

Git Revert

Create a Git repository named **Lab_Q4** and create a branch named **main**. Create a file named **Exam_details.txt** containing your **PRN** number and an initial message, then save it as the **first commit**. Modify the file by adding your **full name** and save the changes as the **second commit**. Further modify the file by adding a **welcome message** and the current date, then save it as the **third commit**. Display the commit history in a compact format and identify the commit sequence. **Revert the last two commits** while preserving the repository history and verify that the file content matches the first commit. Finally, display the complete **commit history** and explain the difference between the original commits and the revert commits.

Step 1: Create a New Git Repository

We create a new Git repository named **Lab_Q4** to track changes made to files and maintain version history.

```
mkdir Lab_Q4 && cd Lab_Q4
```

```
git init
```

```
Lenovo@theMist MINGW64 /d/learn/git/revert
$ mkdir Lab_Q4 && cd Lab_Q4

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4
$ git init
Initialized empty Git repository in D:/learn/git/revert/Lab_Q4/.git/
```

Explanation of attributes

- `mkdir Lab_Q4` → Creates a new directory named Lab_Q4.
- `cd Lab_Q4` → Moves into the project directory.
- `git init` → Initializes an empty Git repository.

Step 2: Create and Switch to the Main Branch

We create a branch named **main** to serve as the primary development branch.

```
git checkout -b main
```

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git checkout -b main
Switched to a new branch 'main'
```

Explanation of attributes

- **checkout** → Switches branches.
- **-b** → Creates a new branch before switching.
- **main** → Name of the new branch.

Step 3: Create the File and Make the First Commit

We create a file containing the PRN number and an initial message. This becomes the baseline version of the project.

File Content

PRN: <PRN_NUMBER>

This line added in first commit

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "first commit"
```

```

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ vim Exam_details_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .
warning: in the working copy of 'Exam_details_260240123060.txt', LF will be replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git commit -m "First commit"
[main (root-commit) d40798c] First commit
1 file changed, 1 insertion(+)
create mode 100644 Exam_details_260240123060.txt

```

Explanation of attributes

- Exam_details_<PRN>.txt → File containing student details.
- git add . → Stages all modified files.
- git commit → Creates a snapshot of the staged changes.
- -m → Specifies the commit message.
- "first commit" → Message describing the commit.

Verify

```

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
This line added in first commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git status
On branch main
nothing to commit, working tree clean

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
d40798c (HEAD -> main) First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git branch
* main

```

Step 4: Modify the File and Make the Second Commit

We update the file by adding the student's full name to demonstrate version tracking.

Updated File Content

PRN: <PRN_NUMBER>

This line added in first commit

Name: <FULL_NAME>

Command

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "second commit"
```

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ vim Exam_details_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .
warning: in the working copy of 'Exam_details_260240123060.txt'
F will be replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git commit -m "second commit"
[main 2b75645] second commit
1 file changed, 2 insertions(+)
```

Explanation of attributes

- `git add .` → Stages the modified file.
- `"second commit"` → Commit message for the second version.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git status
On branch main
nothing to commit, working tree clean

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
2b75645 (HEAD -> main) Second commit
d40798c First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
PRN: 260240123060
This line added in first commit
Name: john j johnson jr.
```

Step 5: Modify the File Again and Make the Third Commit

We add a welcome message and the current date to create another version of the file.

Updated File Content

PRN: <PRN_NUMBER>

This line added in first commit

Name: <FULL_NAME>

Welcome to Ditiss

Date: <CURRENT_DATE>

Command

vim Exam_details_<PRN>.txt

git add .

git commit -m "third commit"

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ vim Exam_details_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .
warning: in the working copy of 'Exam_details_260240123060.txt',
F will be replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git commit -m "Third commit"
[main 5f2ebd2] Third commit
1 file changed, 1 insertion(+)
```

Explanation of attributes

- "third commit" → Commit message identifying the third version.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
5f2ebd2 (HEAD -> main) Third commit
2b75645 Second commit
d40798c First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
PRN: 260240123060
This line added in first commit
Name: john j johnson jr.
Welcom to DITISS
```

Step 6: Display Compact Commit History

We view the commit sequence to identify all changes made to the repository.

Command

git log --oneline

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
5f2ebd2 (HEAD -> main) Third commit
2b75645 Second commit
d40798c First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log
commit 5f2ebd25e5d96153e2f35ae0c3f471e3d1b9a8e2 (HEAD -> main)
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:36:26 2026 +0530

    Third commit

commit 2b75645305730d2d298d04c2ee82c7333c173438
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:33:09 2026 +0530

    Second commit

commit d40798c1330a7b15e66f34fcb2d40d3c985d7acc
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:28:05 2026 +0530

    First commit
```

Explanation of attributes

- **log** → Displays commit history.
- **--oneline** → Shows each commit in a single compact line.

Step 7: Revert the Third Commit

We undo the changes introduced in the third commit while preserving the repository history.

Command

```
git revert HEAD
```

(CLOSE EDITOR IF OPPEND :wq or close editor)

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git revert HEAD
[main be005a3] Revert "Third commit"
1 file changed, 1 deletion(-)
```

Explanation of attributes

- **revert** → Creates a new commit that reverses a previous commit.
- **HEAD** → Refers to the latest commit.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
be005a3 (HEAD -> main) Revert "Third commit"
5f2ebd2 Third commit
2b75645 Second commit
d40798c First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
PRN: 260240123060
This line added in first commit
Name: john j johnson jr.
```


Step 8: Revert the Second Commit

What are we doing and why?

We undo the changes introduced in the second commit, returning the file to the state of the first commit.

Command

```
git revert HEAD~2
```

(CLOSE EDITOR IF OPEND :wq or close editor)

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git revert HEAD~2
[main 7470a51] Revert "Second commit"
1 file changed, 2 deletions(-)
```

Explanation of attributes

- **HEAD~2** → Refers to the commit two positions before the current HEAD.
- **revert** → Preserves history while reversing changes.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log --oneline
7470a51 (HEAD -> main) Revert "Second commit"
be005a3 Revert "Third commit"
5f2ebd2 Third commit
2b75645 Second commit
d40798c First commit

Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
This line added in first commit
```

Step 9: Verify File Content

We confirm that only the original content from the first commit remains.

Command

```
cat Exam_details_<PRN>.txt
```

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ cat Exam_details_260240123060.txt
This line added in first commit
```

- `cat` → Displays the file content on the terminal.

Step 10: Display Complete Commit History

We verify that the repository history contains the original commits along with the revert commits.

Command

```
git log
```

```
Lenovo@theMist MINGW64 /d/learn/git/revert/Lab_Q4 (main)
$ git log
commit 7470a51eb0d99230e540239f70702d0e3ee61309 (HEAD -> main)
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:41:20 2026 +0530

    Revert "Second commit"

    This reverts commit 2b75645305730d2d298d04c2ee82c7333c173438.

commit be005a377a19abf003bdf1bc3f572ca7a61bb8a1
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:39:00 2026 +0530

    Revert "Third commit"

    This reverts commit 5f2ebd25e5d96153e2f35ae0c3f471e3d1b9a8e2.

commit 5f2ebd25e5d96153e2f35ae0c3f471e3d1b9a8e2
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:36:26 2026 +0530

    Third commit

commit 2b75645305730d2d298d04c2ee82c7333c173438
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:33:09 2026 +0530

    Second commit

commit d40798c1330a7b15e66f34fcb2d40d3c985d7acc
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 22:28:05 2026 +0530

    First commit
```

Explanation of attributes

- **log** → Displays detailed commit history including author, date, and commit messages.
-

Expected Outcome

- Repository **Lab_Q4** is successfully created.
- Three commits are created with incremental modifications.
- Commit history is displayed in both compact and detailed formats.
- The second and third commits are reverted.
- File content matches the first commit.
- Repository history contains both original commits and revert commits, demonstrating Git's non-destructive rollback mechanism.

```
mkdir Lab_Q4
```

```
cd Lab_Q4
```

```
git init
```

```
git checkout -b main
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "first commit"
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "second commit"
```

```
vim Exam_details_<PRN>.txt
```

```
git add .
```

```
git commit -m "third commit"
```

```
git log --oneline
```

```
git revert HEAD
```

```
git revert HEAD~2
```

```
cat Exam_details_<PRN>.txt
```

```
git log --oneline
```

```
git log
```

-----EXTRA-----

`git revert <1st\_commit\_id>` does NOT take repository back to first commit state.

It actually undoes the changes introduced by the first commit itself.

So if first commit created `first.txt`, reverting it will delete `first.txt`.

To restore repository to the state of first commit while keeping history, we need to revert the commits that came AFTER it.

Example:

* 1st commit → first.txt

* 2nd commit → adds second.txt

* 3rd commit → adds third.txt

Current state = all 3 files.

Correct method:

```
```bash id="i3h6rk"
```

```
git revert <3rd_commit_id>
```

```
git revert <2nd_commit_id>
```

```
...
```

After reverting 3rd commit:

- \* third.txt removed

- \* repo becomes same as 2nd commit

After reverting 2nd commit:

- \* second.txt removed

- \* repo becomes same as 1st commit

Final state:

```
```bash id="mqgjra"
```

```
first.txt
```

```
...
```

History remains preserved because revert creates new undo commits instead of deleting old commits.

```
mistofVON@ubuntu24:~/git_sce_1$ ls
first.txt  second.txt  third.txt
mistofVON@ubuntu24:~/git_sce_1$ git log --oneline
2268faa (HEAD -> master) 3rd commit
45e0ea2 2nd commit
60e5ab6 1st commit
mistofVON@ubuntu24:~/git_sce_1$ git revert HEAD
[master 7cd7789] Revert "3rd commit"
 1 file changed, 1 deletion(-)
 delete mode 100644 third.txt
mistofVON@ubuntu24:~/git_sce_1$ ^C
mistofVON@ubuntu24:~/git_sce_1$ ls
first.txt  second.txt
mistofVON@ubuntu24:~/git_sce_1$ git revert HEAD~2
[master d319069] Revert "2nd commit"
 1 file changed, 1 deletion(-)
 delete mode 100644 second.txt
mistofVON@ubuntu24:~/git_sce_1$ ls
first.txt
mistofVON@ubuntu24:~/git_sce_1$ git log --oneline
d319069 (HEAD -> master) Revert "2nd commit"
7cd7789 Revert "3rd commit"
2268faa 3rd commit
45e0ea2 2nd commit
60e5ab6 1st commit
mistofVON@ubuntu24:~/git_sce_1$
```